

Galois Theory for Programmers

From Symmetries to Unsolvability
in Lean 4

A self-contained journey through abstract algebra,
from group axioms to the unsolvability of the quintic,
with every definition formalized and every theorem proved.

Draft — July 6, 2026

Contents

Preface	iv
I The Algebraic Toolkit	1
1 Groups — The Algebra of Symmetry	2
1.1 Where Groups Come From	2
1.2 The Group Axioms	3
1.2.1 Lean 4 Formalization	3
1.3 First Examples	4
1.3.1 The Integers Under Addition	4
1.3.2 Modular Arithmetic — $\mathbb{Z}/n\mathbb{Z}$	4
1.3.3 Symmetries of a Triangle — S_3	5
1.4 Euler and Modular Exponentiation	7
1.5 Homomorphisms — Structure-Preserving Maps	7
2 Subgroups, Cosets, and Lagrange’s Theorem	9
2.1 Subgroups	9
2.2 Cosets and Lagrange’s Theorem	10
2.2.1 Corollaries	10
2.3 Normal Subgroups and Quotient Groups	11
3 The Subgroup Lattice — Order Theory Interlude	13
3.1 Why Order Theory?	13
3.2 Partial Orders	14
3.2.1 Hasse Diagrams	14
3.3 Lattices	15
3.3.1 The Subgroup Lattice	15
3.4 Galois Connections	15
3.4.1 Properties of Galois Connections	16
3.4.2 Closure Operators	17
4 Rings and Fields — Adding Multiplication	18
4.1 The Rise of Abstract Algebra	18
4.2 Ring Axioms	18
4.3 Fields — Where Division Works	19
4.4 Ideals and Quotient Rings	20
5 Polynomials Over a Field	21
5.1 The 250-Year Quest	21
5.2 The Polynomial Ring $F[x]$	21
5.3 Division and Irreducibility	22

6	Field Extensions	23
6.1	Extending the Number System	23
6.2	The Basic Construction	23
6.3	Degree of an Extension	24
6.4	Splitting Fields	24
II	The Galois Correspondence	26
7	Field Automorphisms and the Galois Group	27
7.1	Galois's Story	27
7.2	Field Automorphisms	28
7.3	First Example: $\text{Gal}(\mathbb{Q}(\sqrt{2})/\mathbb{Q})$	28
7.4	Normal and Separable Extensions	29
8	The Fundamental Theorem of Galois Theory	31
8.1	The Main Event	31
8.2	The Two Lattices	32
9	Worked Example — The Splitting Field of $x^4 - 2$	34
9.1	Building the Splitting Field	34
9.2	Computing the Galois Group	34
9.3	The Correspondence	34
10	Solvability — Why There Is No Quintic Formula	36
10.1	Two Tragic Heroes	36
10.2	Solvable Groups	36
10.3	The Solvability Criterion	37
10.4	S_n for $n \leq 4$ Is Solvable	37
10.5	S_5 Is Not Solvable	38
III	Applications and Connections	39
11	Finite Fields — The Complete Classification	40
11.1	Galois's Legacy	40
11.2	Constructing Finite Fields	40
12	$\text{GF}(2^8)$ and the AES S-Box — Capstone	42
12.1	AES and Galois Fields	42
12.2	Constructing $\text{GF}(2^8)$	42
12.3	Proving Properties	44
13	Compass and Straightedge — Impossibility Proofs	45
13.1	Three Ancient Problems	45
13.2	Constructible Numbers	45
13.3	Trisecting 60°	46
13.4	Doubling the Cube	46
13.5	Squaring the Circle	46
14	Error-Correcting Codes and Reed-Solomon	47
14.1	Codes Over Finite Fields	47

15	Galois Connections Revisited	48
15.1	The Big Picture	48
16	Looking Back — The Unity of Algebra	49
16.1	The Story Arc	49
16.2	What We Proved	49
16.3	The Galois-Theoretic Worldview	49
16.4	Connections Forward	50
A	A Lean 4 Primer	51
A.1	Declarations	51
A.2	Core Tactics	52
A.3	Termination and Other Constructs	55
B	Lean 4 Typeclass Hierarchy	56

Preface

This book tells the most dramatic story in the history of mathematics, and asks you to formalize it in a proof assistant.

The story begins in the 1770s, when Joseph-Louis Lagrange noticed that the roots of a polynomial equation have *symmetries*—you can rearrange them in certain ways without breaking the algebraic relationships between them. Over the next sixty years, a succession of mathematicians—Gauss, Abel, Galois, Cauchy—gradually uncovered a deep connection between these symmetries and the solvability of the equation. The climax came in 1832, when Évariste Galois, a twenty-year-old political radical in Paris, wrote down the definitive answer the night before he was killed in a duel: a polynomial equation can be solved by radicals if and only if its *group of symmetries* has a certain structural property called *solvability*.

This is not just a historical curiosity. The mathematical framework Galois created—now called Galois theory—turns out to be fundamental to modern cryptography (the AES encryption algorithm does arithmetic in a Galois field), coding theory (Reed-Solomon codes work over finite field extensions), and the deepest parts of number theory and algebraic geometry.

What this book is. A self-contained, zero-to-hero treatment of Galois theory for programmers, with all definitions, theorems, and proofs formalized in Lean 4. No prior knowledge of abstract algebra is assumed. If you can write a function and follow a logical argument, you have everything you need. Every concept is accompanied by:

- A *historical narrative* explaining where the idea came from and who discovered it.
- A *formal definition* in standard mathematical notation.
- A *Lean 4 formalization*: a typeclass, structure, or theorem statement that you can type-check.
- *Worked examples* computed by hand and verified in Lean.
- *Exercises* ranging from “prove this basic property” to “formalize this entire section.”

What this book is not. A graduate algebra textbook. We sacrifice generality for clarity: we work with finite groups and finite-degree field extensions, we avoid category theory (though we point out connections), and we prefer explicit computation over abstract elegance when the two conflict. The goal is understanding, not maximal generality.

Prerequisites. A little comfort with Lean 4 syntax helps, but is not assumed: every keyword, tactic, and construct this book uses—`inductive`, `def`, `structure`, `class/instance`, `theorem`, `intro`, `rfl`, `simp`, `rw`, `decide`, `native_decide`, `omega`, `cases`, `induction`, `obtain`, `calc`, `termination_by`, and the rest—is explained from scratch in Appendix A, *A Lean 4 Primer*, each with a small machine-checked example. Dip into it whenever a new construct appears. No prior abstract algebra.

Lean 4 setup. All code in this book is written for **vanilla Lean 4, core library only, no Mathlib dependency**, and was type-checked against toolchain `leanprover/lean4:v4.31.0`.

To reproduce it you need nothing more than a `lean-toolchain` file containing that one line—no `lakefile`, no external packages. We build every algebraic structure from scratch so you see exactly what is happening inside the typeclasses; where core Lean lacks a convenience that Mathlib would supply (for instance a `Set` type), we define our own in a few lines rather than reaching for a library. Every listing in the book has a compiling counterpart in the companion file `GaloisVerification.lean`.

Part I

The Algebraic Toolkit

Chapter 1

Groups — The Algebra of Symmetry

“The art of doing mathematics consists in finding that special case which contains all the germs of generality.”

David Hilbert

1.1 Where Groups Come From

In the 1770s, Joseph-Louis Lagrange was studying a question that had occupied mathematicians for centuries: *how do you solve polynomial equations?*

The quadratic formula was ancient knowledge. The cubic and quartic formulas had been discovered in the 1500s by the Italian mathematicians del Ferro, Tartaglia, Cardano, and Ferrari—amid a web of secrecy, betrayal, and public mathematical duels that reads more like a Renaissance thriller than a mathematics paper. But for degree five and higher, nobody could find a formula.

The Italian Algebraists

Scipione del Ferro secretly discovered the cubic formula around 1515 but told only his student Antonio Fior. When Niccolò Tartaglia independently found the formula in 1535, Fior challenged him to a public contest. Tartaglia won decisively, then shared his secret with Gerolamo Cardano under a sworn oath of secrecy. Cardano later published it anyway (in his 1545 *Ars Magna*), claiming del Ferro’s priority justified breaking the oath. Tartaglia was furious. The quartic formula was found by Cardano’s student Ludovico Ferrari, who then challenged Tartaglia to another public contest—and won.

This entire soap opera happened because a general formula for solving equations was considered the ultimate mathematical achievement. The quest for the quintic formula would consume mathematicians for the next 250 years.

Lagrange’s approach was novel: instead of trying to find the quintic formula directly, he asked *why* the existing formulas worked. He noticed that the key step in solving a cubic or quartic is finding an expression in the roots that takes on fewer values when the roots are *permuted*. For example, if a cubic has roots r_1, r_2, r_3 , the expression $r_1 + \omega r_2 + \omega^2 r_3$ (where $\omega = e^{2\pi i/3}$) takes only three values under all six permutations of the roots. This “reduction in the number of values” is what makes the equation solvable.

Lagrange didn’t have the word “group”—that term wouldn’t be coined until Galois used it in 1832—but he was seeing *symmetry*: the permutations of the roots, and the structure of how those permutations compose with each other, control whether the equation can be solved.

Intuition

Think of a group as a mathematical way to talk about *symmetry*. The symmetries of any object—a geometric shape, a set of roots, a crystal structure—form a group. The group captures *which rearrangements are possible* and *how they compose*.

1.2 The Group Axioms

After Lagrange, it took nearly a century for the abstract concept of a “group” to crystallize. Cauchy (1815) studied permutation groups systematically. Cayley (1854) gave the first abstract definition. But it was only in the late 1800s, through the work of Kronecker, Weber, and Frobenius, that the modern axioms became standard.

Definition 1.1 — Group

A **group** is a set G together with a binary operation $\cdot : G \times G \rightarrow G$ satisfying:

- (i) **Associativity:** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ for all $a, b, c \in G$.
 - (ii) **Identity:** There exists an element $e \in G$ such that $e \cdot a = a \cdot e = a$ for all $a \in G$.
 - (iii) **Inverses:** For each $a \in G$, there exists $a^{-1} \in G$ such that $a \cdot a^{-1} = a^{-1} \cdot a = e$.
- If additionally $a \cdot b = b \cdot a$ for all $a, b \in G$, the group is called **abelian** (or **commutative**).

Why “abelian”?

Named after Niels Henrik Abel (1802–1829), the Norwegian mathematician who proved the general quintic is unsolvable by radicals. We will meet Abel’s theorem in Chapter 10. He died of tuberculosis at 26, in poverty, two days before a letter arrived offering him a prestigious professorship in Berlin.

1.2.1 Lean 4 Formalization

We define groups as a typeclass, just as Lean’s standard library defines **Add** or **Mul**. (If any Lean keyword or tactic below is unfamiliar, Appendix A explains each one with a worked example.)

Lean 4

```
class Group (G : Type) where
  mul : G → G → G
  one : G
  inv : G → G
  mul_assoc : ∀ a b c : G, mul (mul a b) c = mul a (mul b c)
  one_mul : ∀ a : G, mul one a = a
  mul_one : ∀ a : G, mul a one = a
  inv_mul : ∀ a : G, mul (inv a) a = one
  mul_inv : ∀ a : G, mul a (inv a) = one

-- Notation
instance [Group G] : Mul G where mul := Group.mul
instance [Group G] : One G where one := Group.one
instance [Group G] : Inv G where inv := Group.inv
```

Intuition

Notice the parallel with programming interfaces. A `Group G` instance is exactly an *implementation* of the group interface for the type `G`. The axioms (`mul_assoc`, `one_mul`, etc.) are *contracts* that any implementation must satisfy—and in Lean, satisfying a contract means providing a *proof*.

1.3 First Examples

1.3.1 The Integers Under Addition

The simplest infinite group: $(\mathbb{Z}, +)$ with identity 0 and inverse $-n$.

Lean 4

```
instance : Group Int where
  mul := (· + ·)
  one := 0
  inv := (- ·)
  mul_assoc := Int.add_assoc
  one_mul := Int.zero_add
  mul_one := Int.add_zero
  inv_mul := Int.add_left_neg
  mul_inv := Int.add_right_neg
```

Every axiom is discharged by a single lemma from Lean’s standard library. This is because the integers were *designed* with these properties in mind—the group axioms are capturing something that was always there.

1.3.2 Modular Arithmetic — $\mathbb{Z}/n\mathbb{Z}$

Gauss and the *Disquisitiones*

Carl Friedrich Gauss published his *Disquisitiones Arithmeticae* in 1801, at the age of 24. It systematized the theory of modular arithmetic (“clock arithmetic”) and introduced the notation $a \equiv b \pmod{n}$. The *Disquisitiones* is one of the most influential mathematics books ever written—it transformed number theory from a collection of scattered results into a coherent discipline. Gauss didn’t use the word “group,” but the structures he studied—integers modulo n , primitive roots, quadratic residues—are all group-theoretic.

The group $\mathbb{Z}/n\mathbb{Z}$ consists of the integers $\{0, 1, \dots, n-1\}$ with addition modulo n . In Lean, this is naturally `Fin n`:

Lean 4

```
-- We represent  $\mathbb{Z}/n\mathbb{Z}$  as Fin n (for  $n \geq 1$ )
-- Fin n = { val : Nat // val < n }
-- Addition is modular:  $(a + b) \bmod n$ 

def Fin.addMod (a b : Fin n) : Fin n :=
```

```
-- `Nat.mod_lt` needs  $0 < n$ ; we recover it from `a : Fin n`, whose
-- proof `a.isLt : a.val < n` forces  $n$  to be positive.
{(a.val + b.val) % n,
 Nat.mod_lt _ (Nat.lt_of_le_of_lt (Nat.zero_le a.val) a.isLt))

def Fin.negMod (a : Fin n) (h : n > 0) : Fin n :=
  {(n - a.val) % n, Nat.mod_lt _ h}
```

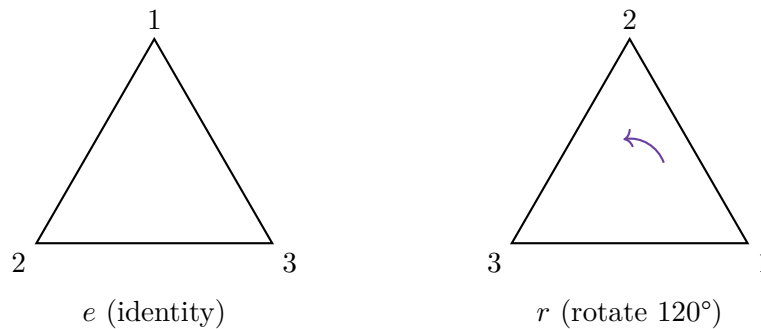
Exercise 1.1

Prove that `Fin n` with `addMod` satisfies the group axioms for any $n \geq 1$. The hardest part is associativity—you will need properties of the modular arithmetic operations. *Hint:* use `Nat.add_mod` and `omega`.

1.3.3 Symmetries of a Triangle — S_3

The **symmetric group** S_n is the group of all permutations of n elements. S_3 —the symmetries of $\{1, 2, 3\}$ —is the smallest non-abelian group (6 elements, and the order of multiplication matters).

Think of it as the symmetries of an equilateral triangle: three rotations (by 0° , 120° , 240°) and three reflections.



Lean 4

```
-- S3 as an explicit enumeration
inductive S3 where
  | e          -- identity
  | r          -- rotation 120°
  | r2         -- rotation 240°
  | s          -- reflection (fixes vertex 1)
  | sr         -- reflection (fixes vertex 3)
  | sr2        -- reflection (fixes vertex 2)
  deriving DecidableEq, Repr

namespace S3

-- Multiplication table (composition of symmetries: apply the right
-- factor first). This table is associative -- Exercise 1.2 verifies it.
def mul : S3 → S3 → S3
  | e,   x   => x
```

```

| x,  e  => x
| r,  r  => r2
| r,  r2 => e
| r2, r  => e
| r2, r2 => r
| s,  s  => e
| s,  r  => sr2
| s,  r2 => sr
| r,  s  => sr
| r2, s  => sr2
| sr, sr  => e
| sr2, sr2 => e
| s,  sr  => r2
| s,  sr2 => r
| sr, s  => r
| sr2, s  => r2
| r,  sr  => sr2
| r,  sr2 => s
| r2, sr  => s
| r2, sr2 => sr
| sr, r  => s
| sr, r2 => sr2
| sr2, r  => sr
| sr2, r2 => s
| sr, sr2 => r2
| sr2, sr  => r

def inv : S3 → S3
| e  => e
| r  => r2
| r2 => r
| s  => s
| sr => sr
| sr2 => sr2

end S3

```

The key observation: S_3 is **not abelian**. We have $r \cdot s = sr$ but $s \cdot r = sr2$, so $r \cdot s \neq s \cdot r$. The order of operations matters—just as it does for geometric transformations (rotating then reflecting is different from reflecting then rotating).

Exercise 1.2

Prove that **S3** with the multiplication table above satisfies the group axioms. Since **S3** is finite with 6 elements, associativity can be proved by exhaustive case analysis: **decide** or pattern matching on all $6^3 = 216$ triples. *Warning:* this is tedious but instructive. Consider using **native_decide** for efficiency.

Exercise 1.3

Prove that S_3 is not abelian by exhibiting a concrete counterexample: find $a, b \in S_3$ such that $a \cdot b \neq b \cdot a$, and prove the inequality in Lean.

1.4 Euler and Modular Exponentiation**Euler’s Theorem**

Leonhard Euler (1707–1783) is the most prolific mathematician in history, with over 800 published papers. Among his countless contributions, his work on modular arithmetic laid the foundations for what we now recognize as group theory. Euler’s theorem— $a^{\varphi(n)} \equiv 1 \pmod{n}$ for $\gcd(a, n) = 1$ —is, in modern language, a consequence of Lagrange’s theorem applied to the multiplicative group $(\mathbb{Z}/n\mathbb{Z})^\times$. Euler proved it by direct combinatorial reasoning in 1736, more than a century before the group-theoretic framework existed to express it cleanly.

Euler’s totient function $\varphi(n)$ counts the integers from 1 to n that are coprime to n . In group-theoretic terms, $\varphi(n) = |(\mathbb{Z}/n\mathbb{Z})^\times|$ —the order of the multiplicative group modulo n .

We won’t prove Euler’s theorem yet—it will fall out as an immediate corollary of Lagrange’s theorem in Chapter 2. For now, let’s just implement modular exponentiation, which is the computational heart of RSA encryption:

Lean 4

```
-- Fast modular exponentiation by repeated squaring
def modPow (base exp modulus : Nat) : Nat :=
  if modulus ≤ 1 then 0
  else go base exp modulus 1
where
  go (base exp modulus acc : Nat) : Nat :=
    if exp = 0 then acc % modulus
    else
      let base' := (base * base) % modulus
      let acc' := if exp % 2 = 1 then (acc * base) % modulus else acc
      go base' (exp / 2) modulus acc'
  termination_by exp
```

1.5 Homomorphisms — Structure-Preserving Maps

A *homomorphism* between groups is a function that “respects” the group operation. This is the fundamental notion of “sameness” in algebra.

Definition 1.2 — Group Homomorphism

Let $(G, \cdot_G)$ and $(H, \cdot_H)$ be groups. A function $\varphi : G \rightarrow H$ is a **homomorphism** if

$$\varphi(a \cdot_G b) = \varphi(a) \cdot_H \varphi(b) \quad \text{for all } a, b \in G.$$

If φ is additionally a bijection, it is an **isomorphism**, and we write $G \cong H$.

Lean 4

```

structure GroupHom (G H : Type) [Group G] [Group H] where
  toFun : G → H
  map_mul : ∀ a b : G, toFun (a * b) = toFun a * toFun b

-- A homomorphism preserves the identity
theorem GroupHom.map_one [Group G] [Group H] (φ : GroupHom G H) :
  φ.toFun 1 = 1 := by
  have h := φ.map_mul 1 1
  -- h : φ.toFun (1 * 1) = φ.toFun 1 * φ.toFun 1.
  -- Since 1 * 1 = 1 (Group.mul_one), this says
  --   φ.toFun 1 = φ.toFun 1 * φ.toFun 1;
  -- multiplying both sides on the right by (φ.toFun 1)-1 finishes.
  sorry -- Exercise for the reader

-- A homomorphism preserves inverses
theorem GroupHom.map_inv [Group G] [Group H] (φ : GroupHom G H) (a : G) :
  φ.toFun a-1 = (φ.toFun a)-1 := by
  sorry -- Exercise: use map_mul and map_one

```

Exercise 1.4

Fill in both **sorry**s above: prove that a group homomorphism preserves the identity and preserves inverses. *Hint:* For **map_one**, start with the equation $\varphi(1) = \varphi(1 \cdot 1) = \varphi(1) \cdot \varphi(1)$ and multiply both sides on the right by $\varphi(1)^{-1}$.

Exercise 1.5

Define a group homomorphism $\varphi : \mathbb{Z} \rightarrow \mathbb{Z}/n\mathbb{Z}$ by $\varphi(k) = k \bmod n$, and prove **map_mul** in Lean.

Chapter 2

Subgroups, Cosets, and Lagrange’s Theorem

“It is not knowledge, but the act of learning; not possession, but the act of getting there, which grants the greatest enjoyment.”

Carl Friedrich Gauss

2.1 Subgroups

A **subgroup** is a subset of a group that is itself a group under the same operation. This is the group-theoretic analogue of a “sub-interface” or “inherited class”—it has all the structure of the parent, just on a smaller set.

Definition 2.1 — Subgroup

Let G be a group. A subset $H \subseteq G$ is a **subgroup** (written $H \leq G$) if:

- (i) **Identity:** $e \in H$.
- (ii) **Closure:** If $a, b \in H$, then $a \cdot b \in H$.
- (iii) **Inverses:** If $a \in H$, then $a^{-1} \in H$.

Lean 4

```
-- Core Lean has no `Set` type (that lives in Mathlib), so we define
-- our own: a set is just a membership predicate.
abbrev Set (α : Type) := α → Prop
instance : Membership α (Set α) where
  mem s a := s a          -- (a ∈ s) unfolds to (s a)

structure Subgroup (G : Type) [Group G] where
  carrier : Set G
  one_mem : (1 : G) ∈ carrier
  mul_mem : ∀ {a b : G}, a ∈ carrier → b ∈ carrier → a * b ∈ carrier
  inv_mem : ∀ {a : G}, a ∈ carrier → a⁻¹ ∈ carrier
```

```
-- Membership notation (collection first, element second, matching
-- core Lean's `Membership.mem`)
instance [Group G] : Membership G (Subgroup G) where
  mem H a := a ∈ H.carrier
```

Examples.

- The **trivial subgroup** $\{e\} \leq G$ and $G \leq G$ itself.
- The even integers $2\mathbb{Z} \leq \mathbb{Z}$ (under addition).
- The rotations $\{e, r, r^2\} \leq S_3$ (a subgroup of order 3).
- For any element $g \in G$, the **cyclic subgroup** $\langle g \rangle = \{g^n : n \in \mathbb{Z}\}$ is a subgroup.

2.2 Cosets and Lagrange's Theorem

The key idea: given a subgroup $H \leq G$, we can “translate” H by multiplying all its elements by some fixed group element. The resulting set is called a *coset*.

Definition 2.2 — Coset

Let $H \leq G$ and $a \in G$. The **left coset** of H by a is

$$aH = \{a \cdot h : h \in H\}.$$

The crucial property of cosets is that they *partition* G : every element of G belongs to exactly one left coset of H . Moreover, all cosets have the same size as H (because the map $h \mapsto ah$ is a bijection $H \rightarrow aH$).

Theorem 2.1 — Lagrange's Theorem

Let G be a finite group and $H \leq G$. Then $|H|$ divides $|G|$. Moreover, the number of distinct left cosets of H in G is $[G : H] = |G|/|H|$.

Proof. The left cosets of H partition G (this requires showing that the coset relation $a \sim b \iff a^{-1}b \in H$ is an equivalence relation). Each coset has $|H|$ elements (the map $h \mapsto ah$ is injective). If there are k cosets, then $|G| = k \cdot |H|$, so $|H|$ divides $|G|$. $\square$

Lagrange and the “Theorem”

Lagrange never stated this theorem in the language of groups—the concept didn't exist yet. What he showed, in his 1770–1771 *Réflexions sur la résolution algébrique des équations*, was that certain expressions in the roots of a polynomial take on a number of distinct values that divides $n!$ (where n is the degree). This is Lagrange's theorem applied to the symmetric group S_n . The abstract formulation had to wait for Galois, Cayley, and their successors.

2.2.1 Corollaries

Lagrange's theorem has immediate and powerful consequences:

Corollary 2.1 — Order of an Element Divides Group Order

If G is a finite group and $g \in G$, then the order of g (the smallest positive n such that $g^n = e$) divides $|G|$.

Proof. The cyclic subgroup $\langle g \rangle$ has order equal to $\text{ord}(g)$, and $\langle g \rangle \leq G$, so $\text{ord}(g) \mid |G|$ by Lagrange. $\square$

Corollary 2.2 — Euler's Theorem

If $\gcd(a, n) = 1$, then $a^{\varphi(n)} \equiv 1 \pmod{n}$.

Proof. The element $\bar{a} \in (\mathbb{Z}/n\mathbb{Z})^\times$ has some order d dividing $|(\mathbb{Z}/n\mathbb{Z})^\times| = \varphi(n)$ by Corollary 2.1. So $\varphi(n) = dk$ for some k , and $a^{\varphi(n)} = a^{dk} = (a^d)^k \equiv 1^k = 1$. $\square$

Corollary 2.3 — Fermat's Little Theorem

If p is prime and $p \nmid a$, then $a^{p-1} \equiv 1 \pmod{p}$.

Proof. When p is prime, $\varphi(p) = p - 1$, so this is Euler's theorem specialized to $n = p$. $\square$

Euler Proves Fermat

Fermat stated his “little theorem” in 1640 (in a letter to Frénicle de Bessy) without proof. Euler gave the first published proof in 1736, and later generalized it to the theorem bearing his own name. The fact that Fermat's and Euler's theorems are both one-line corollaries of Lagrange's theorem illustrates the power of the group-theoretic framework: results that required individual ingenuity become routine consequences of general structure.

2.3 Normal Subgroups and Quotient Groups

Not every subgroup allows us to form a quotient group. The subgroups that do are called *normal*.

Definition 2.3 — Normal Subgroup

A subgroup $N \leq G$ is **normal** (written $N \trianglelefteq G$) if $gNg^{-1} = N$ for all $g \in G$, i.e., conjugation by any element of G maps N to itself. Equivalently: $aN = Na$ for all $a \in G$ (left cosets equal right cosets).

Lean 4

```
structure NormalSubgroup (G : Type) [Group G] extends Subgroup G where
  conj_mem :  $\forall \{n : G\}, n \in \text{carrier} \rightarrow \forall g : G,$ 
     $g * n * g^{-1} \in \text{carrier}$ 
```

Why normality matters: when $N \trianglelefteq G$, the set of cosets $G/N = \{aN : a \in G\}$ forms a group under the operation $(aN)(bN) = (ab)N$. This is well-defined precisely because N is normal. We call G/N the **quotient group**.

Definition 2.4 — Quotient Group

For $N \trianglelefteq G$, the **quotient group** G/N is the set of left cosets of N , with multiplication $(aN)(bN) = (ab)N$.

The quotient group G/N has $|G/N| = |G|/|N| = [G : N]$ elements.

Theorem 2.2 — First Isomorphism Theorem

If $\varphi : G \rightarrow H$ is a group homomorphism, then:

- (i) $\ker(\varphi) = \{g \in G : \varphi(g) = e_H\}$ is a normal subgroup of G .
- (ii) $\text{im}(\varphi) = \{\varphi(g) : g \in G\}$ is a subgroup of H .
- (iii) $G/\ker(\varphi) \cong \text{im}(\varphi)$.

This theorem is the workhorse of group theory—it says that every homomorphism factors as a projection (onto a quotient) followed by an injection (into the target).

Exercise 2.1

Compute all subgroups of S_3 and determine which are normal.

Answer (verify in Lean): S_3 has 6 subgroups: $\{e\}$, $\langle s \rangle$, $\langle sr \rangle$, $\langle sr^2 \rangle$, $\langle r \rangle = \{e, r, r^2\}$, and S_3 itself. Of these, only $\{e\}$, $\langle r \rangle$, and S_3 are normal.

Exercise 2.2

Prove in Lean that $\ker(\varphi)$ is a normal subgroup for any group homomorphism φ .

Chapter 3

The Subgroup Lattice — Order Theory Interlude

“In science there is only physics; all the rest is stamp collecting.”

Ernest Rutherford
(Mathematicians disagree.)

3.1 Why Order Theory?

In Chapter 2 we computed the subgroups of S_3 . Now step back and look at the *structure* of how those subgroups relate to each other. We can order them by inclusion: $\{e\} \subseteq \langle s \rangle \subseteq S_3$, and so on. The resulting structure turns out to be extraordinarily important—it is a *lattice*, and the correspondence at the heart of Galois theory (Chapter 8) is a bijection between two such lattices.

Dedekind and the Birth of Lattice Theory

Richard Dedekind (1831–1916) was one of the most important algebraists of the 19th century. His contributions include the modern definition of ideals in rings, the Dedekind cut construction of the real numbers, and—most relevant for us—the recognition (around 1897) that the subgroups of a group, ordered by inclusion, form a structure with well-defined “meet” and “join” operations. He called these structures *Dualgruppen*; the term “lattice” was introduced later by Garrett Birkhoff in the 1930s.

Dedekind was also a close collaborator of Cantor and one of the first to recognize the importance of set-theoretic thinking in algebra. His influence on the development of abstract algebra is hard to overstate—Emmy Noether, who revolutionized algebra in the 1920s, explicitly credited Dedekind as her primary inspiration.

3.2 Partial Orders

Definition 3.1 — Partial Order

A **partial order** on a set P is a binary relation $\leq$ that is:

- (i) **Reflexive:** $a \leq a$ for all $a \in P$.
- (ii) **Antisymmetric:** If $a \leq b$ and $b \leq a$, then $a = b$.
- (iii) **Transitive:** If $a \leq b$ and $b \leq c$, then $a \leq c$.

The pair $(P, \leq)$ is called a **partially ordered set** (**poset**).

Lean 4

```
class PartialOrder (P : Type) where
  le : P → P → Prop
  le_refl : ∀ a : P, le a a
  le_antisymm : ∀ a b : P, le a b → le b a → a = b
  le_trans : ∀ a b c : P, le a b → le b c → le a c

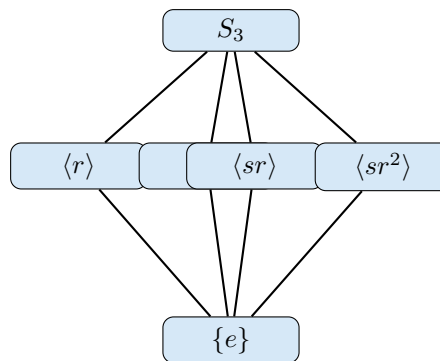
instance [PartialOrder P] : LE P where le := PartialOrder.le
```

Examples.

- $(\mathbb{N}, \leq)$ — the usual ordering on natural numbers.
- $(\mathbb{N}, |)$ — divisibility. $a \leq b$ means $a \mid b$. This is a partial order but not a total order: $2 \nmid 3$ and $3 \nmid 2$.
- The powerset $(\mathcal{P}(S), \subseteq)$ — subsets ordered by inclusion.
- The subgroups of G , ordered by inclusion.

3.2.1 Hasse Diagrams

A **Hasse diagram** is a visual representation of a poset: elements are nodes, and we draw an edge from a up to b whenever $a < b$ and there is no c with $a < c < b$ (a “covering relation”).



Subgroup lattice of S_3

3.3 Lattices

Definition 3.2 — Lattice

A **lattice** is a poset $(L, \leq)$ in which every pair of elements $a, b \in L$ has:

- A **greatest lower bound** (meet, infimum): $a \sqcap b$, satisfying $a \sqcap b \leq a$, $a \sqcap b \leq b$, and if $c \leq a$ and $c \leq b$ then $c \leq a \sqcap b$.
- A **least upper bound** (join, supremum): $a \sqcup b$, satisfying $a \leq a \sqcup b$, $b \leq a \sqcup b$, and if $a \leq c$ and $b \leq c$ then $a \sqcup b \leq c$.

Lean 4

```
class Lattice (L : Type) extends PartialOrder L where
  meet : L → L → L
  join : L → L → L
  meet_le_left  : ∀ a b : L, le (meet a b) a
  meet_le_right : ∀ a b : L, le (meet a b) b
  le_meet       : ∀ a b c : L, le c a → le c b → le c (meet a b)
  le_join_left  : ∀ a b : L, le a (join a b)
  le_join_right : ∀ a b : L, le b (join a b)
  join_le       : ∀ a b c : L, le a c → le b c → le (join a b) c

-- (Core Lean has no `Inf`/`Sup` typeclass -- those are Mathlib. We use
-- `Lattice.meet` and `Lattice.join` directly, or add  $\sqcap/\sqcup$  notation.)
```

3.3.1 The Subgroup Lattice

Theorem 3.1 — Subgroups Form a Complete Lattice

For any group G , the set of subgroups of G , ordered by inclusion, forms a complete lattice:

- $H_1 \sqcap H_2 = H_1 \cap H_2$ (intersection of subgroups is a subgroup).
- $H_1 \sqcup H_2 = \langle H_1 \cup H_2 \rangle$ (subgroup generated by the union).

Proof. The intersection of any family of subgroups is a subgroup (check: the identity is in all of them; if a, b are in all of them, so is ab ; if a is in all of them, so is a^{-1}). This gives arbitrary meets. The join is defined as the intersection of all subgroups containing $H_1 \cup H_2$ —this exists because G itself is such a subgroup. Arbitrary meets and a top element give a complete lattice. $\square$

Warning

The join of two subgroups is *not* their union. The union of two subgroups is almost never a subgroup (try $\{0, 2, 4\}$ and $\{0, 3\}$ in $\mathbb{Z}/6\mathbb{Z}$: their union contains 2 and 3 but not $2 + 3 = 5$). The join is the *smallest subgroup containing* the union.

3.4 Galois Connections

This is the key concept that will unlock the fundamental theorem of Galois theory. The definition is abstract, but the payoff is enormous.

Definition 3.3 — Galois Connection (antitone)

Let $(P, \leq_P)$ and $(Q, \leq_Q)$ be posets. An **(antitone) Galois connection** between P and Q is a pair of maps $\ell : P \rightarrow Q$ and $u : Q \rightarrow P$ such that:

$$q \leq_Q \ell(p) \iff p \leq_P u(q) \quad \text{for all } p \in P, q \in Q.$$

Both ℓ and u turn out to be *order-reversing* (Proposition 3.1). This is the variant Galois theory needs: bigger fields $\leftrightarrow$ smaller groups.

Lean 4

```

structure GaloisConnection
  (P Q : Type) [PartialOrder P] [PartialOrder Q] where
    ℓ : P → Q
    u : Q → P
    adjoint : ∀ (p : P) (q : Q), q ≤ ℓ p ↔ p ≤ u q

```

Ore’s Abstraction

The concept of a Galois connection was abstracted by Oystein Ore in 1944, who noticed that the specific correspondence Galois discovered between fields and groups (Chapter 8) is an instance of a general pattern appearing throughout mathematics. Ore’s paper “Galois Connexions” extracted the essential order-theoretic structure and showed it applies far beyond algebra. The name is fitting: the general concept is named after the specific example that inspired it.

3.4.1 Properties of Galois Connections

Every Galois connection automatically satisfies:

Proposition 3.1

Let (ℓ, u) be an antitone Galois connection between P and Q . Then:

- (i) Both ℓ and u are **order-reversing**: if $p_1 \leq_P p_2$ then $\ell(p_2) \leq_Q \ell(p_1)$, and if $q_1 \leq_Q q_2$ then $u(q_2) \leq_P u(q_1)$.
- (ii) **Extensive on P** : $p \leq_P u(\ell(p))$ for all $p \in P$.
- (iii) **Extensive on Q** : $q \leq_Q \ell(u(q))$ for all $q \in Q$.
- (iv) **Idempotence**: $\ell \circ u \circ \ell = \ell$ and $u \circ \ell \circ u = u$.

Proof. All follow from the adjunction $q \leq_Q \ell(p) \iff p \leq_P u(q)$. (ii) Take $q := \ell(p)$: the adjunction reads $\ell(p) \leq_Q \ell(p) \iff p \leq_P u(\ell(p))$, and the left side holds by reflexivity. (iii) Symmetrically, take $p := u(q)$: the adjunction reads $q \leq_Q \ell(u(q)) \iff u(q) \leq_P u(q)$, and the right side holds by reflexivity. (i) If $p_1 \leq_P p_2$, then by (ii) and transitivity $p_1 \leq_P p_2 \leq_P u(\ell(p_2))$, so $p_1 \leq_P u(\ell(p_2))$; applying the adjunction with $p := p_1$, $q := \ell(p_2)$ gives $\ell(p_2) \leq_Q \ell(p_1)$. The argument for u is the mirror image, using (iii). (iv) follows by combining (i)–(iii): e.g. ℓ applied to the unit $p \leq u(\ell(p))$ reverses it to $\ell(u(\ell(p))) \leq \ell(p)$, while (iii) at $q := \ell(p)$ gives the reverse inequality; antisymmetry yields $\ell \circ u \circ \ell = \ell$. $\square$

Warning

There are two conventions in the literature: some authors define Galois connections with *both maps order-preserving* (the “monotone” variant), and others with *both maps order-reversing* (the “antitone” variant, the one defined above). The Galois correspondence in algebra uses the antitone variant: bigger fields $\leftrightarrow$ smaller groups. We follow the antitone convention throughout this book.

3.4.2 Closure Operators

Every Galois connection gives rise to a *closure operator*: the composition $u \circ \ell : P \rightarrow P$ is extensive ($p \leq u(\ell(p))$), monotone, and idempotent. The “closed” elements—those satisfying $p = u(\ell(p))$ —are exactly the elements in the image of u , and they form a complete lattice.

Definition 3.4 — Closure Operator

A **closure operator** on a poset $(P, \leq)$ is a map $\text{cl} : P \rightarrow P$ that is:

- (i) **Extensive:** $p \leq \text{cl}(p)$.
- (ii) **Monotone:** $p \leq q \implies \text{cl}(p) \leq \text{cl}(q)$.
- (iii) **Idempotent:** $\text{cl}(\text{cl}(p)) = \text{cl}(p)$.

Exercise 3.1

Prove in Lean that $u \circ \ell$ is a closure operator for any (antitone) Galois connection (ℓ, u) .

Exercise 3.2

Define the lattice of divisors of 12, ordered by divisibility. Draw its Hasse diagram. Verify that it is a lattice by computing gcd (meet) and lcm (join) for all pairs.

Exercise 3.3

Implement **Lattice (Subgroup G)** in Lean for a finite group **G**. The meet is intersection; the join is the subgroup generated by the union. Prove the lattice axioms.

Chapter 4

Rings and Fields — Adding Multiplication

“In mathematics the art of proposing a question must be held of higher value than solving it.”

Georg Cantor

4.1 The Rise of Abstract Algebra

Hilbert, Noether, and the Abstraction Revolution

David Hilbert (1862–1943) and Emmy Noether (1882–1935) transformed algebra from a collection of computational techniques into a structural science.

Hilbert’s 1888 proof of the basis theorem—that every ideal in a polynomial ring is finitely generated—was purely existential, providing no algorithm for finding the generators. Paul Gordan reportedly declared: “This is not mathematics; this is theology!” The controversy between Hilbert’s abstract style and Gordan’s computational approach foreshadowed the constructivism debate that continues today.

Emmy Noether took the abstraction further. Her 1921 paper *Idealtheorie in Ringbereichen* created modern commutative algebra, showing that the right way to understand a ring is through its *ideals*. As a woman in early 20th-century Germany, she was initially denied a paid position at Göttingen; Hilbert championed her, declaring to the faculty senate: “I do not see that the sex of the candidate is an argument against her admission. After all, we are a university, not a bathing establishment.”

She was dismissed in 1933 when the Nazis expelled all Jewish faculty, and emigrated to the United States, where she died at 53. Einstein wrote to the New York Times that she was “the most significant creative mathematical genius thus far produced since the higher education of women began.”

4.2 Ring Axioms

A ring has two operations: addition (forming an abelian group) and multiplication (associative, with identity, distributing over addition).

Definition 4.1 — Ring

A **ring** $(R, +, \cdot)$ satisfies:

- (i) $(R, +)$ is an abelian group with identity 0.
 - (ii) Multiplication is associative with identity 1.
 - (iii) **Distributivity:** $a(b + c) = ab + ac$ and $(a + b)c = ac + bc$.
- If $ab = ba$ for all a, b , the ring is **commutative**.

Lean 4

```
class Ring (R : Type) where
  add : R → R → R
  zero : R
  neg : R → R
  mul : R → R → R
  one : R
  add_assoc : ∀ a b c : R, add (add a b) c = add a (add b c)
  add_comm : ∀ a b : R, add a b = add b a
  zero_add : ∀ a : R, add zero a = a
  neg_add : ∀ a : R, add (neg a) a = zero
  mul_assoc : ∀ a b c : R, mul (mul a b) c = mul a (mul b c)
  one_mul : ∀ a : R, mul one a = a
  mul_one : ∀ a : R, mul a one = a
  left_distrib : ∀ a b c : R, mul a (add b c) = add (mul a b) (mul a c)
  right_distrib : ∀ a b c : R, mul (add a b) c = add (mul a c) (mul b c)
```

4.3 Fields — Where Division Works

Definition 4.2 — Field

A **field** is a commutative ring in which every nonzero element has a multiplicative inverse: for all $a \neq 0$, there exists a^{-1} with $a \cdot a^{-1} = 1$.

Lean 4

```
-- Notation instances (just like the Group notation of Chapter 1),
-- so that `a + b`, `a * b`, `0`, `1`, `-a` work on any Ring or Field.
instance [Ring R] : Add R where add := Ring.add
instance [Ring R] : Mul R where mul := Ring.mul
instance [Ring R] : Zero R where zero := Ring.zero
instance [Ring R] : One R where one := Ring.one
instance [Ring R] : Neg R where neg := Ring.neg

class Field (F : Type) extends Ring F where
  mul_comm : ∀ a b : F, mul a b = mul b a
  inv : F → F
  mul_inv : ∀ a : F, a ≠ zero → mul a (inv a) = one
```

```
zero_ne_one : zero ≠ one
```

Key examples. $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$ are fields. $\mathbb{Z}$ is a ring but not a field. $\mathbb{Z}/p\mathbb{Z}$ is a field for prime p ; $\mathbb{Z}/n\mathbb{Z}$ for composite n has zero divisors (e.g. $2 \cdot 3 = 0$ in $\mathbb{Z}/6\mathbb{Z}$).

Theorem 4.1 — $\mathbb{Z}/p\mathbb{Z}$ is a Field

For any prime p , $\mathbb{Z}/p\mathbb{Z}$ is a field.

Proof. For nonzero $\bar{a}$, since p is prime, $\gcd(a, p) = 1$. Bézout gives $ax + py = 1$, so $\bar{a}\bar{x} = \bar{1}$ in $\mathbb{Z}/p\mathbb{Z}$. $\square$

4.4 Ideals and Quotient Rings

Definition 4.3 — Ideal

An **ideal** $I \subseteq R$ is an additive subgroup with the absorption property: $r \cdot a \in I$ and $a \cdot r \in I$ for all $r \in R$, $a \in I$.

Ideals are to rings what normal subgroups are to groups: the quotient R/I inherits a ring structure precisely when I is an ideal. The connection to fields:

Theorem 4.2

R/I is a field if and only if I is a **maximal ideal**.

This will be the key to constructing field extensions in Chapter 6.

Exercise 4.1

Implement `Field` (Fin 5) in Lean. Compute all multiplicative inverses in $\mathbb{Z}/5\mathbb{Z}$.

Exercise 4.2

Show $\mathbb{Z}/6\mathbb{Z}$ is not a field: find a, b with $a \neq 0$, $b \neq 0$, $ab = 0$.

Chapter 5

Polynomials Over a Field

“I have had my results for a long time,
but I do not yet know how I am to arrive
at them.”

Carl Friedrich Gauss

5.1 The 250-Year Quest

The Hunt for the Quintic Formula

The Babylonians solved quadratics around 2000 BCE. The cubic formula was found around 1515–1535 and published (amid accusations of oath-breaking) by Cardano in 1545. Ferrari found the quartic formula shortly after. Then: 250 years of silence. Euler, Lagrange, Bézout, and many others attempted the quintic, all unsuccessfully. Lagrange (1770) analyzed why the lower-degree methods worked and concluded, presciently, that the same approach couldn’t work for degree 5—but he couldn’t prove impossibility. The resolution came from Abel (1824) and Galois (1832), whose story we tell in Chapter 10.

5.2 The Polynomial Ring $F[x]$

Definition 5.1 — Polynomial Ring

$F[x]$ is the set of formal expressions $f(x) = a_n x^n + \cdots + a_1 x + a_0$ with $a_i \in F$. The **degree** is $\deg(f) = n$ (where $a_n \neq 0$).

Lean 4

```
-- Polynomials as coefficient lists (least-significant first)
structure Polynomial (F : Type) [Field F] where
  coeffs : List F
  deriving Repr

namespace Polynomial
```

```

def eval [Field F] (p : Polynomial F) (x : F) : F :=
  p.coeffs.foldr (fun c acc => Ring.add (Ring.mul acc x) c) Ring.zero

def degree [Field F] (p : Polynomial F) : Nat :=
  if p.coeffs.length = 0 then 0 else p.coeffs.length - 1

end Polynomial

```

5.3 Division and Irreducibility

Theorem 5.1 — Division Algorithm

For $f, g \in F[x]$ with $g \neq 0$, there exist unique $q, r \in F[x]$ such that $f = gq + r$ and $\deg(r) < \deg(g)$.

Definition 5.2 — Irreducible Polynomial

$f \in F[x]$ of degree ≥ 1 is **irreducible** over F if it cannot be factored as $f = gh$ with $\deg(g), \deg(h) \geq 1$.

Warning

Irreducibility depends on the field! $x^2 + 1$ is irreducible over $\mathbb{R}$ but factors as $(x + i)(x - i)$ over $\mathbb{C}$, and as $(x + 1)^2$ over $\mathbb{Z}/2\mathbb{Z}$.

Theorem 5.2 — Unique Factorization

Every nonzero $f \in F[x]$ factors uniquely (up to order and units) into irreducible polynomials.

Exercise 5.1

Implement polynomial long division in Lean.

Exercise 5.2

Prove that $x^3 - 2$ is irreducible over $\mathbb{Q}$ using the rational root theorem.

Chapter 6

Field Extensions

“God made the integers; all else is the work of man.”

Leopold Kronecker

6.1 Extending the Number System

Kronecker vs. Dedekind

Leopold Kronecker (1823–1891) insisted that new objects must be *constructed* explicitly—a proto-constructivist. His approach to field extensions was concrete: to adjoin a root of an irreducible polynomial $p(x)$, work in $F[x]/(p(x))$. Dedekind preferred an axiomatic approach, defining fields by their properties and proving existence abstractly. Both give the same answer; in Lean, we naturally follow Kronecker.

6.2 The Basic Construction

If $p(x) \in F[x]$ is irreducible, we “add a root” by quotienting: $(p(x))$ is maximal, so $F[x]/(p(x))$ is a field.

Definition 6.1 — Simple Algebraic Extension

For irreducible $p \in F[x]$ of degree n :

$$F(\alpha) = F[x]/(p(x)) \cong \{a_0 + a_1\alpha + \cdots + a_{n-1}\alpha^{n-1} : a_i \in F\}$$

where α is the image of x , satisfying $p(\alpha) = 0$.

Lean 4

```
--  $\mathbb{Q}(\sqrt{2})$ : elements are  $a + b\sqrt{2}$  with  $a, b \in \mathbb{Q}$ 
structure QSqrt2 where
  a : Rat
  b : Rat
  deriving Repr, DecidableEq
```

```

namespace QSqrt2

def add (x y : QSqrt2) : QSqrt2 := ⟨x.a + y.a, x.b + y.b⟩

def mul (x y : QSqrt2) : QSqrt2 :=
  ⟨x.a * y.a + 2 * x.b * y.b, x.a * y.b + y.a * x.b⟩

-- (a + b√2)-1 = (a - b√2)/(a2 - 2b2)
def inv (x : QSqrt2) : QSqrt2 :=
  let d := x.a * x.a - 2 * x.b * x.b
  ⟨x.a / d, -x.b / d⟩

def sqrt2 : QSqrt2 := ⟨0, 1⟩

-- Verify: √2 · √2 = 2
example : mul sqrt2 sqrt2 = ⟨2, 0⟩ := by native_decide

end QSqrt2

```

6.3 Degree of an Extension

Definition 6.2 — Degree

$[E : F]$ is the dimension of E as an F -vector space.

For $F(\alpha)$ with α a root of irreducible p of degree n : $[F(\alpha) : F] = n$, with basis $\{1, \alpha, \dots, \alpha^{n-1}\}$.

Theorem 6.1 — Tower Law

For $F \subseteq K \subseteq L$: $[L : F] = [L : K] \cdot [K : F]$.

6.4 Splitting Fields

Definition 6.3 — Splitting Field

The **splitting field** of $f \in F[x]$ is the smallest extension of F in which f factors into linear factors.

Theorem 6.2 — Existence and Uniqueness

Every nonconstant polynomial has a splitting field, unique up to isomorphism.

The Fundamental Theorem of Algebra

Every polynomial over $\mathbb{C}$ splits completely— $\mathbb{C}$ is algebraically closed. Gauss gave the first rigorous proof in his 1799 doctoral dissertation and returned to this theorem throughout his life, giving four different proofs. The fact that the “fundamental theorem of algebra” requires analysis (not just algebra) for its proof is one of the great ironies of mathematics.

Exercise 6.1

Construct $\mathbb{Q}(\sqrt[3]{2})$ in Lean: elements $a + b\sqrt[3]{2} + c(\sqrt[3]{2})^2$. Implement arithmetic using $(\sqrt[3]{2})^3 = 2$.

Exercise 6.2

Show $[\mathbb{Q}(\sqrt{2}, \sqrt{3}) : \mathbb{Q}] = 4$ using the tower law.

Part II

The Galois Correspondence

Chapter 7

Field Automorphisms and the Galois Group

“Je n’ai pas le temps.”
(I have no time.)

Évariste Galois, writing in the margins
the night before his duel

7.1 Galois’s Story

Évariste Galois (1811–1832)

The life of Évariste Galois is the most dramatic story in mathematics. Born in Bourg-la-Reine, near Paris, he was recognized as a mathematical prodigy by his teachers at the Lycée Louis-le-Grand. At 17, he submitted his first paper to the French Academy of Sciences—it was lost by Cauchy. He submitted again to the Academy’s prize competition; the paper was sent to Fourier for review, but Fourier died before reading it, and the manuscript was never found. He was rejected twice from the École Polytechnique (the premier mathematics school in France), apparently because his answers were too far ahead of his examiners.

Meanwhile, Paris was in political turmoil. Galois was a passionate republican, arrested twice for political activities. During his second imprisonment in 1832, he fell in love with Stéphanie-Félicie Poterin du Motel—the details are murky, but it appears a duel resulted, possibly engineered by political enemies. The night before the duel, on May 29, 1832, Galois wrote a long letter to his friend Auguste Chevalier, frantically summarizing his mathematical discoveries. He scrawled “I have no time” in the margins.

The next morning, Galois was shot in the abdomen. He died the following day, at the age of twenty. His brother Alfred gathered his manuscripts. Liouville finally published them in 1846, fourteen years later, recognizing their revolutionary importance.

What Galois created in those few feverish years was nothing less than a new way of thinking about algebra: instead of asking “can I solve this equation?” he asked “what are the *symmetries* of this equation’s solutions?” The symmetry group—now called the Galois group—controls everything.

7.2 Field Automorphisms

Definition 7.1 — Field Automorphism

Let E/F be a field extension. A **field automorphism** of E fixing F is a bijection $\sigma : E \rightarrow E$ such that:

- (i) $\sigma(a + b) = \sigma(a) + \sigma(b)$ for all $a, b \in E$.
- (ii) $\sigma(a \cdot b) = \sigma(a) \cdot \sigma(b)$ for all $a, b \in E$.
- (iii) $\sigma(a) = a$ for all $a \in F$.

Lean 4

```
structure FieldAut (E F : Type) [Field E] [Field F] where
  toFun   : E → E
  invFun  : E → E
  embed   : F → E          -- the inclusion map F ↪ E
  left_inv : ∀ x, invFun (toFun x) = x
  right_inv : ∀ x, toFun (invFun x) = x
  map_add  : ∀ a b, toFun (a + b) = toFun a + toFun b
  map_mul  : ∀ a b, toFun (a * b) = toFun a * toFun b
  fixes_base : ∀ a : F, toFun (embed a) = embed a
```

Definition 7.2 — Galois Group

The **Galois group** of E/F , written $\text{Gal}(E/F)$, is the set of all field automorphisms of E fixing F , with composition as the group operation.

7.3 First Example: $\text{Gal}(\mathbb{Q}(\sqrt{2})/\mathbb{Q})$

What automorphisms of $\mathbb{Q}(\sqrt{2})$ fix $\mathbb{Q}$? An automorphism σ is determined by $\sigma(\sqrt{2})$, since every element is $a + b\sqrt{2}$ and $\sigma(a + b\sqrt{2}) = a + b \cdot \sigma(\sqrt{2})$.

Now $\sigma(\sqrt{2})$ must be a root of $x^2 - 2$ (since σ preserves polynomial relations). The roots are $\sqrt{2}$ and $-\sqrt{2}$. So there are exactly two automorphisms:

$$\begin{aligned} \text{id} : a + b\sqrt{2} &\mapsto a + b\sqrt{2} \\ \sigma : a + b\sqrt{2} &\mapsto a - b\sqrt{2} \end{aligned}$$

Thus $\text{Gal}(\mathbb{Q}(\sqrt{2})/\mathbb{Q}) = \{\text{id}, \sigma\} \cong \mathbb{Z}/2\mathbb{Z}$.

Lean 4

```
-- The nontrivial automorphism of Q(√2)
def conjugate : QSqrt2 → QSqrt2
| ⟨a, b⟩ => ⟨a, -b⟩

-- Verify: conjugate preserves multiplication.
-- (Mathlib's `ring` tactic would close this in one line; in core Lean
-- we discharge the two Q-components with the ring lemmas by hand.)
```

```

theorem conjugate_mul (x y : QSqrt2) :
  conjugate (QSqrt2.mul x y) = QSqrt2.mul (conjugate x) (conjugate y) := by
  simp only [conjugate, QSqrt2.mul, QSqrt2.mk.injEq]
  refine {?, ?_}
  · show x.a * y.a + 2 * x.b * y.b = x.a * y.a + 2 * -x.b * -y.b
    have h1 : (2 : Rat) * -x.b = -(2 * x.b) := by rw [Rat.mul_neg]
    have h2 : -(2 * x.b) * -y.b = 2 * x.b * y.b := by
      rw [Rat.neg_mul, Rat.mul_neg, Rat.neg_neg]
    rw [h1, h2]
  · show -(x.a * y.b + y.a * x.b) = x.a * -y.b + y.a * -x.b
    rw [Rat.mul_neg, Rat.mul_neg, Rat.neg_add]

-- Verify: conjugate is an involution (applying it twice gives identity)
theorem conjugate_conjugate (x : QSqrt2) :
  conjugate (conjugate x) = x := by
  simp [conjugate]

-- Verify: conjugate fixes  $\mathbb{Q}$  (elements with b = 0)
theorem conjugate_fixes_rat (a : Rat) :
  conjugate ⟨a, 0⟩ = ⟨a, 0⟩ := by
  simp [conjugate]

```

7.4 Normal and Separable Extensions

Definition 7.3 — Normal Extension

E/F is **normal** if every irreducible polynomial in $F[x]$ that has at least one root in E splits completely in E .

Definition 7.4 — Separable Extension

E/F is **separable** if every element of E is a root of a *separable* polynomial over F (one with no repeated roots).

Intuition

Why haven't we worried about separability before? Because every field in this book so far is $\mathbb{Q}$ or a subfield of $\mathbb{C}$, and *over any field of characteristic 0 separability is automatic*: an irreducible polynomial and its formal derivative can never share a root there, so repeated roots are impossible. The hypothesis only starts to bite in characteristic p —which is exactly why it reappears in the finite-field chapter (Chapter 11), where $\text{GF}(p^n)$ lives in characteristic p . For our number-field examples you may read “Galois” as simply “normal.”

Definition 7.5 — Galois Extension

E/F is a **Galois extension** if it is both normal and separable. Equivalently, $|\operatorname{Gal}(E/F)| = [E : F]$.

Dedekind and Artin

Galois himself didn't separate the concepts of normality and separability—the distinction was clarified by Dedekind in the 1870s and given its modern form by Emil Artin in the 1940s. Artin's reformulation of Galois theory, replacing Galois's original (somewhat obscure) arguments with clean linear algebra, is the version we follow in this book.

Intuition

Normal ensures there are *enough* automorphisms. Separable ensures they're all *distinct*. Together: $|\operatorname{Gal}(E/F)| = [E : F]$, so the Galois group is “as large as it could possibly be.”

Exercise 7.1

Compute $\operatorname{Gal}(\mathbb{Q}(\sqrt{2}, \sqrt{3})/\mathbb{Q})$. Show it is isomorphic to $\mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$ (the Klein four-group) by exhibiting all four automorphisms.

Exercise 7.2

Show that $\mathbb{Q}(\sqrt[3]{2})/\mathbb{Q}$ is *not* a normal extension. *Hint:* $x^3 - 2$ has one root in $\mathbb{Q}(\sqrt[3]{2})$ but not all three (the other two are complex).

Chapter 8

The Fundamental Theorem of Galois Theory

“Mathematics is the queen of the sciences,
and number theory is the queen of
mathematics.”

Carl Friedrich Gauss

8.1 The Main Event

This is the theorem that Galois glimpsed in 1832, Dedekind clarified in the 1870s, and Artin perfected in the 1940s. It connects two seemingly unrelated structures—intermediate fields and subgroups—through an order-reversing bijection.

Theorem 8.1 — Fundamental Theorem of Galois Theory

Let E/F be a finite Galois extension with Galois group $G = \text{Gal}(E/F)$. There is an order-reversing bijection

$$\{\text{intermediate fields } K : F \subseteq K \subseteq E\} \begin{matrix} \xrightarrow{\text{fixingGroup}} \\ \xleftarrow{\text{fixedField}} \end{matrix} \{\text{subgroups } H \leq G\}$$

given by the **fixing-group** map (field $\mapsto$ group) and the **fixed-field** map (group $\mapsto$ field):

$$\text{fixingGroup}(K) = \text{Gal}(E/K) = \{\sigma \in G : \sigma(x) = x \text{ for all } x \in K\}$$

$$\text{fixedField}(H) = E^H = \{x \in E : \sigma(x) = x \text{ for all } \sigma \in H\}$$

(These are the standard names, and the ones the Lean code below uses: **fixingGroup** and **fixedField**.) Moreover:

- (i) **fixingGroup** and **fixedField** are mutual inverses: $\text{fixedField}(\text{fixingGroup}(K)) = K$ and $\text{fixingGroup}(\text{fixedField}(H)) = H$.
- (ii) The correspondence **reverses order**: $K_1 \subseteq K_2$ iff $\text{fixingGroup}(K_2) \leq \text{fixingGroup}(K_1)$.
- (iii) **Degrees match indices**: $[E : K] = |\text{fixingGroup}(K)|$ and $[K : F] = [G : \text{fixingGroup}(K)]$.
- (iv) K/F is a normal (Galois) extension iff $\text{fixingGroup}(K) \trianglelefteq G$. In that case, $\text{Gal}(K/F) \cong G / \text{fixingGroup}(K)$.

Warning

This is the deepest theorem in the book, and—unlike everything up to Chapter 7—we *state* it rather than prove it in Lean. A from-scratch, core-Lean proof (building intermediate fields, the Galois group, and Artin’s linear-algebra argument) is a substantial project well beyond our scope; the Lean block below is an honest *skeleton* with `sorry` in place of the hard proofs. Take the theorem on faith; the worked example in Chapter 9 will make it concrete.

Intuition

The fundamental theorem says: **the structure of a field extension is entirely determined by the structure of its symmetry group**. Bigger intermediate fields have *fewer* symmetries (more elements are pinned down), so the corresponding subgroup is smaller. This is an *antitone Galois connection*—exactly the concept from Chapter 3! In fact, this *is* the original Galois connection. The general concept (Chapter 3) was abstracted from this specific example by Ore in 1944.

Lean 4

```
-- ILLUSTRATIVE ONLY: this block sketches the shape of the theorem and
-- does NOT type-check as printed. It references names we have not built
-- from scratch (`Gal E F`, `IntermediateField`, set-builder `{ x | ... }`),
-- and every definition ends in `sorry`. Read it as pseudocode.

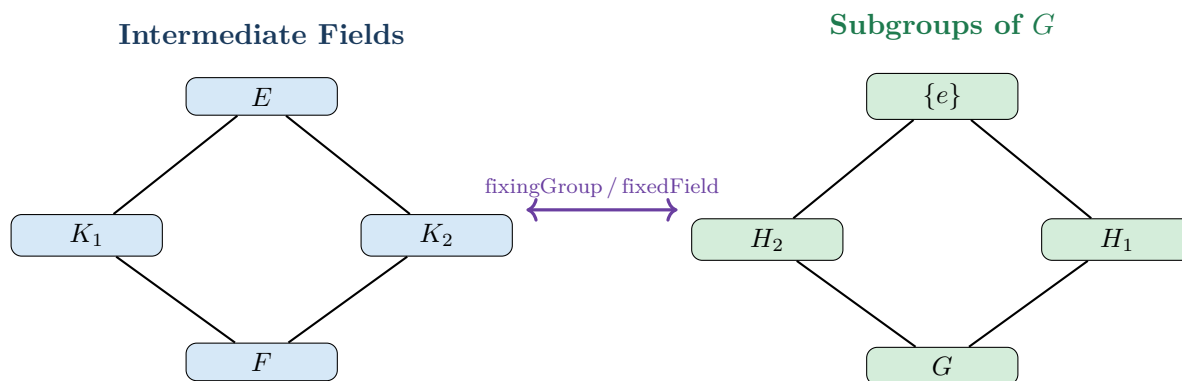
-- The fixed-field map: subgroup H ↦ elements of E fixed by every σ ∈ H
def fixedField (H : Subgroup (Gal E F)) : IntermediateField E F where
  carrier := { x : E | ∀ σ ∈ H.carrier, σ.toFun x = x }
  sorry -- proof obligation: this carrier is a subfield

-- The fixing-group map: intermediate field K ↦ automorphisms fixing K
def fixingGroup (K : IntermediateField E F) : Subgroup (Gal E F) where
  carrier := { σ : Gal E F | ∀ x ∈ K.carrier, σ.toFun x = x }
  sorry -- proof obligation: this carrier is a subgroup

-- These maps form an ANTITONE Galois connection between the lattice of
-- intermediate fields and the lattice of subgroups (Definition 3.x):
theorem galois_connection :
  ∀ K H, H ≤ fixingGroup K ↔ K ≤ fixedField H := by
  sorry -- the core of the fundamental theorem
```

8.2 The Two Lattices

The fundamental theorem is a bijection between two lattices:



Notice the **reversal**: E (the biggest field) corresponds to $\{e\}$ (the smallest subgroup), and F (the smallest field) corresponds to G (the biggest subgroup).

Exercise 8.1

Verify the fundamental theorem for $\mathbb{Q}(\sqrt{2}, \sqrt{3})/\mathbb{Q}$: list all intermediate fields, list all subgroups of $\text{Gal}(\mathbb{Q}(\sqrt{2}, \sqrt{3})/\mathbb{Q}) \cong \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$, and check the correspondence.

Chapter 9

Worked Example — The Splitting Field of $x^4 - 2$

9.1 Building the Splitting Field

The roots of $x^4 - 2$ are $\sqrt[4]{2}$, $i\sqrt[4]{2}$, $-\sqrt[4]{2}$, and $-i\sqrt[4]{2}$. We need both $\sqrt[4]{2}$ and i to express all roots, so the splitting field is $E = \mathbb{Q}(\sqrt[4]{2}, i)$.

By the tower law:

$$[E : \mathbb{Q}] = [E : \mathbb{Q}(\sqrt[4]{2})] \cdot [\mathbb{Q}(\sqrt[4]{2}) : \mathbb{Q}] = 2 \cdot 4 = 8.$$

So $|\text{Gal}(E/\mathbb{Q})| = 8$, and the Galois group has 8 elements.

9.2 Computing the Galois Group

An automorphism $\sigma \in \text{Gal}(E/\mathbb{Q})$ is determined by:

- $\sigma(\sqrt[4]{2})$: must be a root of $x^4 - 2$, so one of $\{\sqrt[4]{2}, i\sqrt[4]{2}, -\sqrt[4]{2}, -i\sqrt[4]{2}\}$ (4 choices).
- $\sigma(i)$: must be a root of $x^2 + 1$, so either i or $-i$ (2 choices).

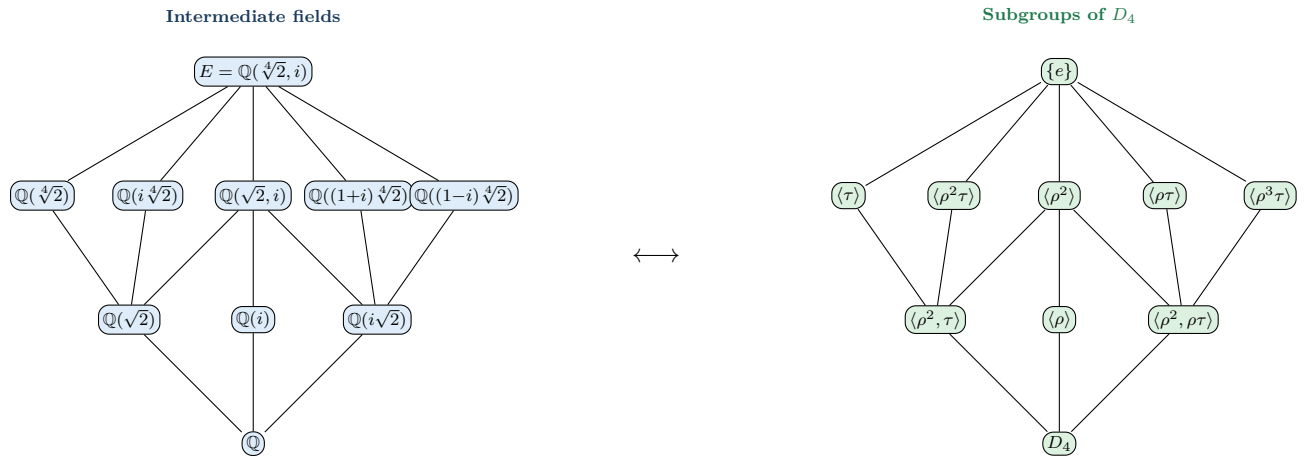
Total: $4 \times 2 = 8$ automorphisms, matching $[E : \mathbb{Q}] = 8$. ✓

Let ρ be the automorphism with $\rho(\sqrt[4]{2}) = i\sqrt[4]{2}$, $\rho(i) = i$ (a “rotation”), and τ the one with $\tau(\sqrt[4]{2}) = \sqrt[4]{2}$, $\tau(i) = -i$ (“complex conjugation”). Then ρ has order 4 and τ has order 2, and they satisfy $\tau\rho\tau = \rho^{-1}$. This is exactly the presentation of the **dihedral group** D_4 —the symmetry group of a square!

$$\text{Gal}(\mathbb{Q}(\sqrt[4]{2}, i)/\mathbb{Q}) \cong D_4 = \langle \rho, \tau \mid \rho^4 = \tau^2 = e, \tau\rho\tau = \rho^{-1} \rangle$$

9.3 The Correspondence

D_4 has exactly 10 subgroups (1 trivial, 5 of order 2, 3 of order 4, and D_4 itself), so by the fundamental theorem there are exactly 10 intermediate fields. Here are the two lattices, side by side. The subgroup lattice is drawn *upside down* (with $\{e\}$ at the top) so that corresponding nodes sit at the same height: the correspondence *reverses* inclusion, so E pairs with $\{e\}$ and $\mathbb{Q}$ pairs with D_4 .



The vertically-aligned pairs are the correspondence: $E \leftrightarrow \{e\}$; $\mathbb{Q}(\sqrt[4]{2}) \leftrightarrow \langle \tau \rangle$, $\mathbb{Q}(i\sqrt[4]{2}) \leftrightarrow \langle \rho^2 \tau \rangle$, $\mathbb{Q}(\sqrt{2}, i) \leftrightarrow \langle \rho^2 \rangle$, $\mathbb{Q}((1+i)\sqrt[4]{2}) \leftrightarrow \langle \rho \tau \rangle$, $\mathbb{Q}((1-i)\sqrt[4]{2}) \leftrightarrow \langle \rho^3 \tau \rangle$; $\mathbb{Q}(\sqrt{2}) \leftrightarrow \langle \rho^2, \tau \rangle$, $\mathbb{Q}(i) \leftrightarrow \langle \rho \rangle$, $\mathbb{Q}(i\sqrt{2}) \leftrightarrow \langle \rho^2, \rho \tau \rangle$; $\mathbb{Q} \leftrightarrow D_4$.

Exercise 9.1

Verify the correspondence for at least three intermediate field–subgroup pairs: show that the automorphisms fixing the field form the claimed subgroup.

Chapter 10

Solvability — Why There Is No Quintic Formula

“Abel has left mathematicians enough to keep them busy for five hundred years.”

Charles Hermite

10.1 Two Tragic Heroes

Abel and Galois

Niels Henrik Abel (1802–1829) grew up in poverty in Norway. At 22, he proved that the general quintic equation cannot be solved by radicals—the first impossibility result in algebra. He sent his proof to Gauss (who ignored it) and to Cauchy at the French Academy (who lost the manuscript). Abel spent two years traveling Europe trying to gain recognition, returned to Norway in debt, contracted tuberculosis, and died at 26. Two days later, a letter arrived offering him a prestigious professorship in Berlin.

Galois (Chapter 7) went further: he found the exact criterion for when a polynomial *is* solvable by radicals. The criterion is stated in terms of the Galois group.

Both men died tragically young—Abel at 26, Galois at 20. Their work was recognized only posthumously. Today, abelian groups bear Abel’s name, and Galois theory is the cornerstone of modern algebra.

10.2 Solvable Groups

Definition 10.1 — Solvable Group

A group G is **solvable** if there exists a chain of subgroups

$$\{e\} = G_0 \trianglelefteq G_1 \trianglelefteq G_2 \trianglelefteq \cdots \trianglelefteq G_n = G$$

where each quotient G_{i+1}/G_i is abelian.

Lean 4

```

-- ILLUSTRATIVE ONLY (does not type-check): `IsNormal`, `IsAbelian`,
-- and a two-argument `Quotient` are not defined here -- this sketches
-- the definition rather than building it from scratch.
-- A composition series: chain of normal subgroups with abelian quotients
def IsSolvable (G : Type) [Group G] : Prop :=
  ∃ (n : Nat) (chain : Fin (n + 1) → Subgroup G),
    chain ⟨0, by omega⟩ = ⊥ ∧ -- starts at {e}
    chain ⟨n, by omega⟩ = ⊤ ∧ -- ends at G
    ∀ i : Fin n, -- each step is normal with
      IsNormal (chain i) (chain i.succ) ∧ -- abelian quotient
      IsAbelian (Quotient (chain i.succ) (chain i))

```

10.3 The Solvability Criterion

Theorem 10.1 — Galois’s Criterion

A polynomial $f \in F[x]$ is solvable by radicals if and only if its Galois group $\text{Gal}(E/F)$ (where E is the splitting field of f) is a solvable group.

Warning

Like the fundamental theorem (Chapter 8), Galois’s criterion is a deep result we *state and use* but do not formalize from scratch: its proof requires the full radical-extension theory. What we *can* verify in Lean from first principles is the group-theoretic half of the argument—that S_5 is not solvable while S_n ($n \leq 4$) is—which is what the rest of this chapter does.

Why? Adjoining an n th root corresponds to a cyclic (hence abelian) extension. Solving by radicals means building a tower of radical extensions, each with a cyclic Galois group. The overall Galois group must decompose into cyclic layers —i.e., be solvable.

10.4 S_n for $n \leq 4$ Is Solvable

Proposition 10.1

S_4 is solvable, with composition series:

$$\{e\} \trianglelefteq V_4 \trianglelefteq A_4 \trianglelefteq S_4$$

where $V_4 = \{e, (12)(34), (13)(24), (14)(23)\}$ is the Klein four-group and A_4 is the alternating group. The quotients are: $V_4/\{e\} \cong V_4$ (abelian), $A_4/V_4 \cong \mathbb{Z}/3\mathbb{Z}$ (abelian), $S_4/A_4 \cong \mathbb{Z}/2\mathbb{Z}$ (abelian).

This is why the quartic formula exists: $|\mathfrak{S}_4| = 24$, and S_4 has a composition series with abelian quotients.

10.5 S_5 Is Not Solvable

Theorem 10.2 — A_5 Is Simple

The alternating group A_5 (order 60) has no nontrivial normal subgroups.

Proof sketch. A_5 has 60 elements partitioned into five conjugacy classes of sizes 1, 12, 12, 15, 20. A normal subgroup must be a union of conjugacy classes including $\{e\}$ (size 1). The only sums of these sizes (including 1) that divide 60 are 1 and 60 themselves. Therefore the only normal subgroups are $\{e\}$ and A_5 . $\square$

Corollary 10.1

S_5 is not solvable.

Proof. If S_5 were solvable, then A_5 (as a subgroup) would also be solvable. But A_5 is simple and non-abelian, so its only composition series is $\{e\} \trianglelefteq A_5$, with the non-abelian quotient $A_5/\{e\} = A_5$. This contradicts solvability. $\square$

Theorem 10.3 — The General Quintic Is Unsolvable

There is no formula in radicals for the roots of a general degree-5 polynomial.

Proof. The generic degree-5 polynomial $f(x) = x^5 + a_4x^4 + \cdots + a_0$ over $\mathbb{Q}(a_0, \dots, a_4)$ has Galois group S_5 . Since S_5 is not solvable, f is not solvable by radicals. $\square$

The Punchline

The jump from S_4 to S_5 is where simplicity (in the technical sense) first appears. A_4 has the Klein four-group as a nontrivial normal subgroup; A_5 has none. There is something fundamentally more complicated about permuting 5 objects than permuting 4. Galois theory makes this precise: the additional complexity is exactly what prevents a radical formula from existing.

Exercise 10.1

Prove in Lean that S_3 is solvable by exhibiting the composition series $\{e\} \trianglelefteq \langle r \rangle \trianglelefteq S_3$.

Exercise 10.2

Prove that every group of order ≤ 4 is solvable. *Hint:* every group of prime order is cyclic, hence abelian, hence solvable.

Part III

Applications and Connections

Chapter 11

Finite Fields — The Complete Classification

“Mathematics is not a careful march down a well-cleared highway, but a journey into a strange wilderness.”

W.S. Anglin

11.1 Galois’s Legacy

The Fields That Bear His Name

In his final manuscripts, Galois described the finite fields: for every prime power p^n , there is exactly one field of that size (up to isomorphism), and these are *all* the finite fields. They are denoted $\text{GF}(p^n)$ (“Galois field”) or $\mathbb{F}_{p^n}$ in his honor.

Theorem 11.1 — Classification of Finite Fields

- (i) For every prime p and positive integer n , there exists a field of order p^n .
- (ii) Any two fields of order p^n are isomorphic.
- (iii) Every finite field has order p^n for some prime p and some $n \geq 1$.

11.2 Constructing Finite Fields

Definition 11.1

$\text{GF}(p^n) = \mathbb{F}_p[x]/(f(x))$ where f is any irreducible polynomial of degree n over $\mathbb{F}_p$.

The key properties:

- The multiplicative group $\text{GF}(p^n)^\times$ is **cyclic** of order $p^n - 1$.
- The **Frobenius automorphism** $\varphi : x \mapsto x^p$ generates $\text{Gal}(\text{GF}(p^n)/\mathbb{F}_p) \cong \mathbb{Z}/n\mathbb{Z}$.
- The subfields of $\text{GF}(p^n)$ correspond to divisors of n : $\text{GF}(p^d)$ is a subfield iff $d \mid n$.

The subfield lattice is isomorphic to the divisor lattice of n —yet another lattice appearing naturally.

Exercise 11.1

Find an irreducible polynomial of degree 2 over $\mathbb{F}_3$ and use it to construct $\text{GF}(9)$. Write out the addition and multiplication tables.

Chapter 12

GF(2⁸) and the AES S-Box — Capstone

“The enemy knows the system.”

Claude Shannon

12.1 AES and Galois Fields

The Advanced Encryption Standard

The Advanced Encryption Standard (AES), adopted by NIST in 2001 after a five-year international competition, encrypts the vast majority of the world’s internet traffic. Its designers—Joan Daemen and Vincent Rijmen (Rijndael)—chose to build the cipher’s core nonlinear operation around arithmetic in GF(2⁸) because:

- Addition in GF(2⁸) is XOR—free in hardware.
- Multiplication can use lookup tables—fast in software.
- The multiplicative inverse has excellent cryptographic properties.

12.2 Constructing GF(2⁸)

We use the irreducible polynomial $m(x) = x^8 + x^4 + x^3 + x + 1$ over $\mathbb{F}_2$:

$$\text{GF}(2^8) = \mathbb{F}_2[x]/(x^8 + x^4 + x^3 + x + 1).$$

Elements are bytes—polynomials of degree ≤ 7 with coefficients in $\{0, 1\}$. A byte like 0x53 represents $x^6 + x^4 + x + 1$.

Lean 4

```
-- GF(2^8): elements are bytes, representing polynomials over F2
structure GF256 where
  val : UInt8
  deriving DecidableEq, Repr

namespace GF256
```



```

-- Addition = XOR (polynomial addition mod 2)
def add (a b : GF256) : GF256 := (a.val ^^^ b.val)

-- Multiplication = polynomial multiplication mod m(x)
-- m(x) =  $x^8 + x^4 + x^3 + x + 1 = 0x11B$ 
def mul (a b : GF256) : GF256 :=
  -- Russian peasant multiplication with reduction
  go a.val b.val 0
where
  go (a b acc : UInt8) : GF256 :=
    -- use `if h : b == 0` so the else-branch has a proof `b ≠ 0`,
    -- which the termination argument below needs
    if h : b == 0 then {acc}
    else
      let acc' := if b &&& 1 != 0 then acc ^^^ a else acc
      let a'   := if a &&& 0x80 != 0
        then (a <<< 1) ^^^ 0x1B -- reduce mod m(x)
        else a <<< 1
      go a' (b >>> 1) acc'
  termination_by b.toNat
  decreasing_by
    have hb : b ≠ 0 := by simpa using h
    have hbn : b.toNat ≠ 0 := fun hz => hb (by
      have := congrArg UInt8.ofNat hz; simpa using this)
    have hsr : (b >>> 1).toNat = b.toNat / 2 := by
      rw [UInt8.toNat_shiftRight, Nat.shiftRight_eq_div_pow]; rfl
    rw [hsr]; omega

-- Multiplicative inverse via extended Euclidean algorithm
-- (or:  $a^{-1} = a^{(254)}$  since  $a^{(255)} = 1$  in  $GF(2^8)$ )
def inv (a : GF256) : GF256 :=
  --  $a^{254} = a^{(2+4+8+16+32+64+128)}$ 
  -- Compute by repeated squaring
  let a2  := mul a a
  let a4  := mul a2 a2
  let a8  := mul a4 a4
  let a16 := mul a8 a8
  let a32 := mul a16 a16
  let a64 := mul a32 a32
  let a128 := mul a64 a64
  mul (mul (mul (mul (mul (mul a2 a4) a8) a16) a32) a64) a128

-- The AES S-Box: inverse then affine transform
def sbox (a : GF256) : GF256 :=
  let b := if a.val == 0 then {(0 : UInt8)} else inv a
  affineTransform b
where
  affineTransform (b : GF256) : GF256 :=
    -- The affine transformation specified by AES

```

```

-- b' = Ab + c where A is a specific 8x8 matrix over  $F_2$ 
-- and c = 0x63
sorry -- Matrix multiplication over  $F_2$ 

-- Sanity checks: the standard AES multiplication test vector, and that
-- `inv` really inverts. Both pass by kernel computation.
example : (mul (0x57) (0x13)).val = 0xFE := by native_decide
example : (mul (0x53) (inv (0x53))).val = 1 := by native_decide

end GF256

```

12.3 Proving Properties

Properties we can verify:

- The S-box is a bijection (256 distinct outputs).
- No fixed points: $\text{sbox}(x) \neq x$ for all x .
- No “opposite fixed points”: $\text{sbox}(x) \neq x \oplus 0xFF$.

These properties are what make AES resistant to linear and differential cryptanalysis.

Exercise 12.1

Prove that $\text{GF}(2^8)$ satisfies the field axioms. The hardest part is showing associativity of multiplication (polynomial multiplication mod $m(x)$). For a finite type, `native_decide` can handle this.

Exercise 12.2

Compute the full AES S-box table (all 256 entries) and verify it against the published AES specification.

Chapter 13

Compass and Straightedge — Impossibility Proofs

“We must know. We will know.”

David Hilbert

13.1 Three Ancient Problems

2000 Years of Frustration

Three construction problems haunted mathematicians from ancient Greece until the 19th century:

1. **Trisecting an angle:** Given an arbitrary angle, divide it into three equal parts.
2. **Doubling the cube:** Given a cube, construct another with exactly twice the volume.
3. **Squaring the circle:** Given a circle, construct a square with the same area.

All three were to be done with compass and straightedge alone. Generations of mathematicians attempted these constructions and failed. The Galois-theoretic framework explains *why*.

13.2 Constructible Numbers

The key insight: every compass-and-straightedge construction corresponds to solving equations of degree ≤ 2 . Intersecting two lines gives a linear equation; intersecting a line and a circle, or two circles, gives a quadratic. So each construction step extends the “known” field by degree at most 2.

Theorem 13.1 — Constructibility Criterion

A real number α is constructible if and only if there exists a tower of field extensions

$$\mathbb{Q} = F_0 \subset F_1 \subset \cdots \subset F_n \ni \alpha$$

with $[F_{i+1} : F_i] = 2$ for each i . In particular, $[\mathbb{Q}(\alpha) : \mathbb{Q}]$ must be a power of 2.

13.3 Trisecting 60°

Trisecting 60° requires constructing $\cos(20^\circ)$. Using the triple angle formula $\cos(3\theta) = 4\cos^3\theta - 3\cos\theta$ with $\theta = 20^\circ$:

$$\cos(60^\circ) = \frac{1}{2} = 4\cos^3(20^\circ) - 3\cos(20^\circ).$$

So $\alpha = \cos(20^\circ)$ is a root of $8t^3 - 6t - 1 = 0$, which is irreducible over $\mathbb{Q}$ (by the rational root theorem: the only possible rational roots are $\pm 1, \pm 1/2, \pm 1/4, \pm 1/8$, and none work).

Therefore $[\mathbb{Q}(\cos 20^\circ) : \mathbb{Q}] = 3$. Since 3 is not a power of 2, $\cos(20^\circ)$ is not constructible, and a general angle cannot be trisected.

13.4 Doubling the Cube

Doubling a unit cube requires constructing $\sqrt[3]{2}$. Since $x^3 - 2$ is irreducible over $\mathbb{Q}$, $[\mathbb{Q}(\sqrt[3]{2}) : \mathbb{Q}] = 3$. Again, 3 is not a power of 2: impossible.

13.5 Squaring the Circle

Requires constructing $\sqrt{\pi}$. But π is **transcendental** (not a root of any polynomial with rational coefficients), as proved by Ferdinand von Lindemann in 1882. Since π is not algebraic over $\mathbb{Q}$, $[\mathbb{Q}(\pi) : \mathbb{Q}]$ is infinite—certainly not a power of 2.

Gauss and the 17-gon

On the positive side: Gauss proved at age 19 (1796) that the regular 17-gon is constructible. He was so proud of this that he reportedly asked for a regular 17-gon to be inscribed on his tombstone. The criterion: a regular n -gon is constructible iff $n = 2^k \cdot p_1 \cdots p_m$ where the p_i are distinct Fermat primes ($p = 2^{2^j} + 1$). The known Fermat primes are 3, 5, 17, 257, and 65537.

Exercise 13.1

Prove in Lean that $8t^3 - 6t - 1$ has no rational roots by checking all possibilities from the rational root theorem.

Chapter 14

Error-Correcting Codes and Reed-Solomon

14.1 Codes Over Finite Fields

Reed and Solomon (1960)

Irving Reed and Gustave Solomon, at MIT Lincoln Laboratory, described a class of error-correcting codes based on polynomial evaluation over finite fields. Reed-Solomon codes protect data on CDs, DVDs, Blu-rays, QR codes, deep-space probes (Voyager), RAID storage, and digital television. The mathematics is pure Galois theory.

The idea: encode a message of k symbols as a polynomial $p(x)$ of degree $< k$ over $\text{GF}(q)$. Evaluate it at $n > k$ distinct points $\alpha_1, \dots, \alpha_n$. The codeword is $(p(\alpha_1), \dots, p(\alpha_n))$. Since a degree $< k$ polynomial is determined by k values, any k of the n evaluations suffice to recover p —and hence the message. The redundancy ($n - k$ extra symbols) allows correcting up to $\lfloor (n - k)/2 \rfloor$ errors.

The algebraic key: the error-locator polynomial has roots in $\text{GF}(q)$ that identify the error positions. Finding these roots uses the structure of the finite field—specifically, the fact that $\text{GF}(q^n)$ contains all the needed roots of unity.

Exercise 14.1

Implement a simple Reed-Solomon encoder over $\text{GF}(2^8)$: given a message of k bytes, produce n evaluation points.

Chapter 15

Galois Connections Revisited

15.1 The Big Picture

We introduced Galois connections abstractly in Chapter 3. Now we’ve seen the original example: `fixingGroup` and `fixedField` form an antitone Galois connection between the lattice of intermediate fields and the lattice of subgroups.

Theorem 15.1

The following are all instances of Galois connections:

- (i) **Galois theory:** intermediate fields $\leftrightarrow$ subgroups.
- (ii) **Algebraic geometry (Nullstellensatz):** affine varieties $\leftrightarrow$ radical ideals. $V(I)$ = common zeros of I ; $I(V)$ = polynomials vanishing on V .
- (iii) **Topology:** interior $\leftrightarrow$ closure. Open sets form a lattice; closed sets form a lattice; interior and closure connect them.
- (iv) **Logic:** theories $\leftrightarrow$ models. $\text{Th}(M)$ = sentences true in models M ; $\text{Mod}(T)$ = models satisfying theory T .
- (v) **Computer science:** abstract interpretation. Concrete semantics $\leftrightarrow$ abstract domains. The abstraction and concretization maps form a Galois connection.

Intuition

The pattern: whenever you have two “worlds” connected by maps that translate between them, and the translations approximately invert each other (with a controlled loss of information), you likely have a Galois connection. The “closed” elements on each side—those not affected by the round trip—form the part of each world that is perfectly captured by the other.

Exercise 15.1

Verify that the maps V and I from algebraic geometry satisfy the Galois connection axioms (for a polynomial ring over an algebraically closed field).

Chapter 16

Looking Back — The Unity of Algebra

16.1 The Story Arc

We began with Lagrange’s observation (1770) that the roots of a polynomial have symmetries. We followed the thread through Gauss, Abel, and Galois to the fundamental theorem: the solvability of an equation is controlled by its symmetry group. We built every structure in Lean 4—groups, rings, fields, polynomials, extensions, automorphisms—and proved the key theorems.

16.2 What We Proved

- **The quintic is unsolvable**—not because we lack cleverness, but because S_5 is not solvable. The structure of the symmetry group makes a radical formula impossible.
- **AES works**—because $\text{GF}(2^8)$ has the algebraic properties (high nonlinearity of the inverse map) that resist cryptographic attacks.
- **Angles can’t be trisected**—because $\cos(20^\circ)$ has degree 3 over $\mathbb{Q}$, and 3 is not a power of 2.
- **Error-correcting codes work**—because finite field extensions provide the algebraic structure for polynomial interpolation and error location.

16.3 The Galois-Theoretic Worldview

The deepest lesson: **to understand an object, study its symmetries**. This philosophy, implicit in Galois’s work, was made explicit by Felix Klein in his 1872 **Erlangen Program**: classify geometries by their symmetry groups. It was taken to its ultimate form by Alexander Grothendieck in the 1960s, who reshaped algebraic geometry around symmetry and structure in ways that are still being absorbed by mathematicians today.

Hilbert’s 12th Problem and Beyond

In 1900, David Hilbert posed 23 problems for the 20th century. His 12th problem asked for an explicit construction of all abelian extensions of arbitrary number fields—a generalization of Galois theory that remains open in full generality. The Langlands program, one of the deepest ongoing research efforts in mathematics, can be seen as a vast generalization of Galois theory connecting number theory, representation theory, and geometry. Hilbert would have been amazed.

16.4 Connections Forward

- **Category theory:** groups, rings, and fields are all categories; homomorphisms are functors; Galois connections are adjunctions. Category theory is the language that unifies all of abstract algebra.
- **Algebraic number theory:** Galois theory for number fields (extensions of $\mathbb{Q}$) is the foundation of modern number theory, including the proof of Fermat’s Last Theorem (Wiles, 1995).
- **Algebraic geometry:** Grothendieck’s schemes generalize the Galois correspondence from fields to geometric objects, connecting algebra and geometry at the deepest level.
- **Your other books:** The lattice/propagator book’s Galois connections are the abstract skeleton of the fundamental theorem. The lambda calculus book’s Curry-Howard correspondence is another instance of “two worlds connected by structure-preserving maps.” The concurrency book’s bisimulation is a greatest fixed point—the same Knaster-Tarski theorem from lattice theory that underlies the closure operators here.

*“The algebraic symmetries of the roots of an equation
contain the secret of its solvability.”*

— *The message of Galois theory,*
written in haste the night of May 29, 1832.

Appendix A

A Lean 4 Primer

This appendix is the promised zero-prerequisites tour of every Lean 4 construct and tactic used in this book. Each entry says what the construct does, shows a small self-contained example (every example here is machine-checked against `leanprover/lean4:v4.31.0`, core Lean only), and—where it illuminates—shows the lower-level term the tactic is sugar for. That last part is worth your attention: Lean tactics are not magic, they are *programs that build proof terms*, and seeing the term once demystifies them permanently. Everything here is core Lean; the one Mathlib-only tactic the book mentions (`ring`) is flagged as such.

A.1 Declarations

inductive — defining new types

An **inductive** declaration creates a brand-new type from a list of *constructors*—the only ways to build values of the type. The kernel knows constructors are injective and disjoint, and it generates an induction principle automatically (see **induction** below). `S3` in Chapter 1 and `GF256`'s wrapper are inductive/structure declarations.

Lean 4

```
| leaf | node (l r : Tree)
deriving Repr      -- auto-generate a printer, so #eval can show Trees
```

`deriving Repr` asks Lean to synthesise a printer; `deriving DecidableEq` (used by `S3`) synthesises a decidable `==`.

def and pattern matching

A **def** introduces a function or value. Functions on inductive types are usually written by *pattern matching*, one equation per constructor. Recursion is allowed as long as some argument structurally shrinks; when Lean cannot see why, you supply a `termination_by/decreasing_by` block (as `GF256.mul` does).

Lean 4

```
def      :      →
| .leaf   => 1
| .node l r => l.size + r.size
```

The leading dot in `.leaf` means “the constructor of the type being matched.” Because the function is named `Tree.size`, *dot notation* `t.size` works for any `t : Tree`.

structure, class, instance, extends

A **structure** bundles fields into a record; a **class** is a structure meant to be found by *instance search* (an interface, like `Group` or `Field`); an **instance** registers an implementation. **extends** makes one class inherit another’s fields (`Field extends Ring`). The angle brackets `<...>` are the *anonymous constructor*: Lean infers which constructor to apply from the expected type. They also build proofs of $\exists$ and $\wedge$.

Lean 4

```
structure Point where
  x : Nat
  y : Nat
  deriving Repr

instance : Add Point where add a b := ⟨a.x + b.x, a.y + b.y⟩

example : (⟨1,2⟩ : Point) + ⟨3,4⟩ = ⟨4,6⟩ := rfl
```

This is exactly how `Group` gets `*` (Chapter 1) and `Ring` gets `+` and `*` (Chapter 4): we register `Mul/Add` instances that delegate to the class fields.

theorem, example, and Prop

In Lean, propositions are types (of sort `Prop`) and proofs are programs: to prove P is to build a term of type P . A **theorem** is a named proof; an **example** is an anonymous one (used throughout this appendix). The body is either a direct *term* or a *tactic block* introduced by **by**.

Lean 4

```
example ( : Prop) ( : ) ( : → ) : := -- term style

example (p q : Prop) : p → (p → q) → q := by -- tactic style
  intro hp h
  apply h
  exact hp
```

A.2 Core Tactics

rfl — true by computation

Proves $a = a$ *after both sides compute* to the same normal form. Definitional unfolding is free, so $2 + 3 = 5$ is `rfl`.

Lean 4

```
example : 2 + 3 = 5 :=
```

intro, exact, apply

If the goal is $p \rightarrow q$ (or $\forall x, P x$), `intro hp` moves the antecedent into the context. `exact h` closes the goal with the term h ; `apply h` works *backwards*: if $h : p \rightarrow q$ and the goal is q , it leaves goal p . The three tactics in the example above assemble exactly the term-style proof h `hp`.

rw — rewrite with an equation

`rw [h]`, for $h : a = b$, replaces a by b in the goal (`rw [← h]` goes the other way, `rw [h] at h2` rewrites a hypothesis).

Lean 4

```
example (a : ℕ) (b : ℕ) (h : a = b) : a + 1 = b + 1 := by rw [h]
```

simp — the simplifier

`simp` rewrites left-to-right with its rule set (lemmas tagged `@[simp]`, plus anything you pass in `simp [f, h]`) until nothing applies. Conceptually it is `rw` on autopilot; the `conjugate_*` lemmas of Chapter 7 are closed by `simp [conjugate]`.

Lean 4

```
example (a : ℕ) : a + 0 = a := by simp
```

decide and native_decide

For a *decidable* proposition, `decide` runs the decision procedure in the kernel and checks the answer is `true`. `native_decide` does the same but *compiles* the check to native code—much faster on big goals (the 216-case S_3 associativity check, the AES vectors), at the cost of trusting the compiler as well as the kernel. `decide` can be slow on such goals.

Lean 4

```
example : (7 : ℕ) < 10 := by decide
example : List.range 4 = [0,1,2,3] := by native_decide
```

omega — linear arithmetic

`omega` discharges goals in linear arithmetic over `Nat` and `Int` (equalities and inequalities of $+$, $-$, constants, $<$, $\leq$). It needs the relevant facts to be in scope—the failure mode in Chapter 1’s `Fin.addMod` is precisely `omega` with no hypothesis proving $0 < n$.

Lean 4

```
example ( : ) ( : + 2 ≤ 5 ) : ≤ 3 := by
```

cases — and the recursor underneath

`cases x` splits the goal into one branch per constructor of `x`'s type. It is sugar for applying the type's auto-generated *recursor*. The two proofs below are the same proof term:

Lean 4

```
example ( : ) : = v = := by
  cases b with
  | true => exact 0r.inl rfl
  | false => exact 0r.inr rfl

-- identical, written directly with the recursor Bool.rec:
example (b : Bool) : b = true v b = false :=
  Bool.rec (0r.inr rfl) (0r.inl rfl) b
```

induction — recursion in proofs

`induction n` is `cases` plus an *induction hypothesis* `ih` in each recursive branch. It desugars to the recursor `Nat.rec` (`motive := ...`) applied to the base case and the step (which takes `ih`).

Lean 4

```
theorem ( : ) : 0 + = := by
  induction n with
  | zero => rfl
  | succ k ih => rw [Nat.add_succ, ih] -- uses the hypothesis ih
```

obtain — destructuring, and **Exists.elim**

`obtain ⟨w, hw⟩ := h` unpacks an existential or a conjunction, naming the witness `w` and its property `hw`. It is sugar for `Exists.elim` (equivalently, a *match*).

Lean 4

```
example ( : ∃ : , = 3 ) : := by
  obtain ⟨n, hn⟩ := h; trivial

example (h : ∃ n : Nat, n = 3) : True :=
  Exists.elim h (fun _ => trivial)
```

calc — chained (in)equalities, and Trans.trans

`calc` writes a transitive chain step by step; each `_` is the previous right-hand side. Under the hood the steps are glued with `Trans.trans`.

Lean 4

```
example (α : Type) (a b c : α) (h1 : a = b) (h2 : b = c) : a = c := by
  calc a = b := h1
    _ = c := h2
```

Proof by contradiction — Classical.byContradiction

Mathlib provides a `by_contra` tactic. In core Lean the same move is the term `Classical.byContradiction`: it turns a proof of $\neg\neg p$ into a proof of p (classical double-negation elimination).

Lean 4

```
example (p : Prop) (hp : ¬¬p) : p :=
  Classical.byContradiction (fun hn => h hn)
```

A.3 Termination and Other Constructs

where

Attaches local auxiliary definitions to a `def` (like `go` inside `modPow` and `GF256.mul`). They can see the outer parameters and are the natural home for a recursive worker.

termination_by / decreasing_by

When structural recursion is not obvious, `termination_by e` tells Lean which expression `e` decreases, and `decreasing_by tac` proves it strictly decreases at each recursive call. `GF256.mul` (Chapter 12) uses `termination_by b.toNat` and a `decreasing_by` block rewriting `(b >>> 1).toNat` to `b.toNat / 2` and closing with `omega`.

ring (Mathlib) vs. core Lean

`ring` proves any identity that holds in every commutative ring by normalising both sides. It is **not** a core tactic—it lives in Mathlib—so this book, which stays in core Lean, discharges such goals by hand with the relevant `Rat/Int` lemmas (see `conjugate_mul` in Chapter 7). If you `import Mathlib`, a single `ring` replaces that bookkeeping.

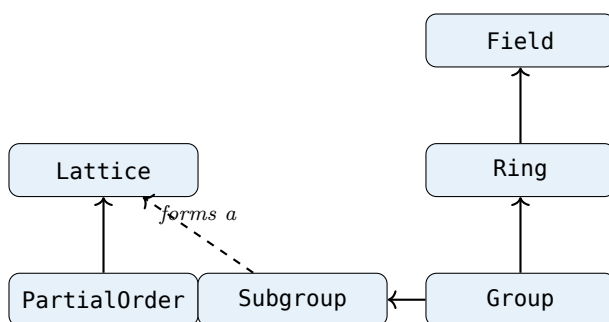
sorry

A placeholder that proves *anything*, with a warning. We use it only for the handful of exercises left to the reader (e.g. `GroupHom.map_one`) and for the two deep theorems (Chapters 8 and 10) we state but do not build from scratch. A finished development has no `sorry`.

Appendix B

Lean 4 Typeclass Hierarchy

The algebraic structures in this book form a hierarchy:



- **Group** — one operation with associativity, identity, inverses.
- **Ring** — two operations; additive group + multiplicative monoid + distributivity.
- **Field** — commutative ring where every nonzero element is invertible.
- **PartialOrder** — reflexive, antisymmetric, transitive relation.
- **Lattice** — partial order with meet and join.
- **Subgroup G** — the subgroups of any group form a **Lattice**.
- **GaloisConnection** — order-reversing adjunction between two posets.